

Методичка 14. Наследование

1. Зачем нужна эта тема

Наследование нужно, когда в программе есть несколько похожих классов.

Например, в игре могут быть разные персонажи:

- воин;
- лучник;
- маг.

У всех них есть общие данные:

- имя;
- здоровье;
- урон;
- уровень.

И общие действия:

- атаковать;
- получать урон;
- проверять, жив ли персонаж.

Но у каждого типа персонажа есть и отличия:

- воин атакует мечом;
- лучник стреляет из лука;
- маг использует заклинание.

Без наследования пришлось бы копировать одинаковый код в каждый класс. Это неудобно: если нужно изменить общую логику, придётся менять её сразу в нескольких местах.

Наследование позволяет вынести общее поведение в базовый класс, а отличия оставить в дочерних классах.

Эта тема продолжает ООП. Чтобы понимать наследование, нужно уже знать классы, объекты, `__init__`, методы и `self`.

2. Главная идея темы

Наследование позволяет создать новый класс на основе уже существующего.

Например:

```
class Unit:
    pass

class Warrior(Unit):
    pass
```

`Warrior` наследуется от `Unit`.

Это значит, что воин получает всё общее, что есть у юнита, и может добавить своё.

Базовый класс — класс, от которого наследуются.

Дочерний класс — класс, который наследуется от базового.

Наследование — механизм, при котором один класс получает поля и методы другого класса.

Переопределение метода — ситуация, когда дочерний класс создаёт свою версию метода, который уже был в базовом классе.

`super()` — способ обратиться к базовому классу из дочернего класса.

3. Базовый синтаксис

Базовый класс

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def print_info(self):
        print("Имя:", self.name)
        print("Здоровье:", self.health)
```

Класс `Unit` описывает общие свойства всех юнитов.

Дочерний класс

```
class Warrior(Unit):  
    pass
```

`Warrior` наследуется от `Unit`.

Теперь у воина есть всё, что было у `Unit`.

```
warrior = Warrior("Воин", 100)  
  
warrior.print_info()
```

Добавление своего метода

```
class Warrior(Unit):  
    def attack(self):  
        print(self.name, "атакует мечом")
```

Теперь у `Warrior` есть метод `attack()`.

Переопределение метода

```
class Unit:  
    def attack(self):  
        print("Юнит атакует")  
  
class Warrior(Unit):  
    def attack(self):  
        print("Воин атакует мечом")  
  
class Archer(Unit):  
    def attack(self):  
        print("Лучник стреляет из лука")
```

У каждого дочернего класса своя версия атаки.

Использование `super`

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

class Warrior(Unit):
    def __init__(self, name, health, armor):
        super().__init__(name, health)
        self.armor = armor
```

`super().__init__(name, health)` вызывает конструктор базового класса.

4. Разбор темы от простого к сложному

Шаг 1. Проблема дублирования

Без наследования:

```
class Warrior:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def is_alive(self):
        return self.health > 0

class Archer:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def is_alive(self):
        return self.health > 0
```

Код повторяется. Поля `name`, `health` и метод `is_alive()` одинаковые.

Шаг 2. Вынос общего в базовый класс

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def is_alive(self):
        return self.health > 0
```

Теперь общее поведение находится в одном месте.

Шаг 3. Создание наследников

```
class Warrior(Unit):
    pass

class Archer(Unit):
    pass
```

`Warrior` и `Archer` получают поля и методы от `Unit`.

```
warrior = Warrior("Воин", 100)
archer = Archer("Лучник", 80)

print(warrior.is_alive())
print(archer.is_alive())
```

Шаг 4. Добавление отличий

```
class Warrior(Unit):
    def attack(self):
        print(self.name, "бьёт мечом")

class Archer(Unit):
    def attack(self):
        print(self.name, "стреляет из лука")
```

Общее остаётся в `Unit`, различия — в дочерних классах.

Шаг 5. Переопределение метода

```
class Unit:
    def attack(self):
        print("Юнит атакует")

class Warrior(Unit):
    def attack(self):
        print("Воин атакует мечом")
```

Если у дочернего класса есть метод с таким же названием, Python вызовет метод дочернего класса.

Шаг 6. Расширение `__init__`

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

class Warrior(Unit):
    def __init__(self, name, health, armor):
        super().__init__(name, health)
        self.armor = armor
```

Воину нужны общие поля `name` , `health` и дополнительное поле `armor` .

Общие поля создаёт базовый класс через `super()` .

Шаг 7. Список разных объектов

```
units = [
    Warrior("Воин", 100, 20),
    Archer("Лучник", 80)
]

for unit in units:
    unit.attack()
```

В списке могут лежать разные объекты, но если у них есть метод `attack()` , их можно обрабатывать одинаково.

Шаг 8. Базовый метод-заглушка

```
class Unit:
    def attack(self):
        raise NotImplementedError("Метод attack нужно переопределить")
```

Так можно показать, что каждый наследник обязан написать свою атаку.

Для школьников это можно понимать так: базовый класс говорит “у каждого юнита должна быть атака, но конкретная атака зависит от типа юнита”.

5. Частые ошибки учеников

Ошибка 1. Копируют одинаковый код вместо наследования

Неправильно:

```
class Warrior:
    def __init__(self, name, health):
        self.name = name
        self.health = health

class Archer:
    def __init__(self, name, health):
        self.name = name
        self.health = health
```

Почему это плохо:

Одинаковый код повторяется в нескольких классах.

Правильно:

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

class Warrior(Unit):
    pass

class Archer(Unit):
    pass
```

Ошибка 2. Забывают указать базовый класс в скобках

Неправильно:

```
class Warrior:
    pass
```

Почему это ошибка:

Так `Warrior` не наследуется от `Unit`.

Правильно:

```
class Warrior(Unit):
    pass
```

Ошибка 3. Забывают вызвать super

Неправильно:

```
class Warrior(Unit):
    def __init__(self, name, health, armor):
        self.armor = armor
```

Почему это ошибка:

Поля `name` и `health` не создаются.

Правильно:

```
class Warrior(Unit):
    def __init__(self, name, health, armor):
        super().__init__(name, health)
        self.armor = armor
```

Ошибка 4. Путают базовый и дочерний класс

Неправильно:

```
unit = Unit("Воин", 100)
unit.attack()
```

Почему это может быть ошибкой:

Если базовый `Unit` нужен только как общая основа, конкретные объекты лучше

создавать через дочерние классы.

Правильно:

```
warrior = Warrior("Воин", 100, 20)
warrior.attack()
```

Ошибка 5. Переопределяют метод случайно

Неправильно:

```
class Unit:
    def print_info(self):
        print(self.name)

class Warrior(Unit):
    def print_info(self):
        print("Воин")
```

Почему это может быть ошибкой:

Метод базового класса заменился. Если нужно было дополнить поведение, стоит использовать `super()`.

Правильно:

```
class Warrior(Unit):
    def print_info(self):
        super().print_info()
        print("Тип: воин")
```

Ошибка 6. Наследуют то, что не является разновидностью

Неправильно:

```
class Sword(Unit):
    pass
```

Почему это плохо:

Меч — это не разновидность юнита. Наследование должно читаться как “является”.

Правильно:

```
class Warrior(Unit):  
    pass
```

Воин является юнитом. Лучник является юнитом. Меч не является юнитом.

Ошибка 7. Делают слишком глубокую цепочку наследования

Неправильно:

```
class Unit:  
    pass  
  
class Human(Unit):  
    pass  
  
class Soldier(Human):  
    pass  
  
class Warrior(Soldier):  
    pass  
  
class FireWarrior(Warrior):  
    pass
```

Почему это сложно:

Чем длиннее цепочка, тем труднее понять, откуда берётся метод или поле.

Для учебных задач лучше использовать простую структуру:

```
class Unit:  
    pass  
  
class Warrior(Unit):  
    pass  
  
class Archer(Unit):  
    pass
```

6. Мини-примеры для закрепления

Пример 1. Базовый класс

```
class Animal:
    def __init__(self, name):
        self.name = name

    def print_name(self):
        print(self.name)
```

Закрепляется создание общего класса.

Пример 2. Наследник

```
class Cat(Animal):
    pass

cat = Cat("Барсик")

cat.print_name()
```

Закрепляется наследование метода.

Пример 3. Метод наследника

```
class Dog(Animal):
    def bark(self):
        print(self.name, "лает")

dog = Dog("Шарик")

dog.bark()
```

Закрепляется добавление своего метода.

Пример 4. Переопределение метода

```
class Animal:
    def speak(self):
        print("Животное издаёт звук")

class Cat(Animal):
    def speak(self):
        print("Мяу")

cat = Cat()

cat.speak()
```

Закрепляется переопределение.

Пример 5. super в `__init__`

```
class Animal:
    def __init__(self, name):
        self.name = name

class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name)
        self.color = color

cat = Cat("Барсик", "рыжий")

print(cat.name)
print(cat.color)
```

Закрепляется вызов конструктора базового класса.

Пример 6. Игровые юниты

```
class Unit:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def is_alive(self):
        return self.health > 0

class Warrior(Unit):
    def attack(self):
        print(self.name, "атакует мечом")

class Archer(Unit):
    def attack(self):
        print(self.name, "стреляет из лука")
```

Закрепляется типичная структура наследования.

Пример 7. Список разных наследников

```
units = [
    Warrior("Воин", 100),
    Archer("Лучник", 80)
]

for unit in units:
    unit.attack()
```

Закрепляется общий интерфейс для разных объектов.

7. Практические задания

Лёгкий уровень

1. Создать базовый класс `Animal` с полем `name`.
2. Создать класс `Cat`, который наследуется от `Animal`.
3. Создать класс `Dog`, который наследуется от `Animal`.
4. Добавить в `Cat` метод `meow()`.
5. Добавить в `Dog` метод `bark()`.

Средний уровень

1. Создать базовый класс `Unit` с полями `name` , `health` .
2. Создать классы `Warrior` и `Archer` , которые наследуются от `Unit` .
3. Добавить каждому классу свой метод `attack()` .
4. Добавить в `Unit` метод `is_alive()` .
5. Создать список из разных юнитов и вызвать у каждого `attack()` .

Повышенный уровень

1. Добавить классу `Warrior` поле `armor` , используя `super()` .
2. Создать базовый класс `Transport` и наследников `Car` , `Bike` , `Bus` , у каждого сделать свой метод `move()` .
3. Создать базовый класс `Courier` и наследников `FootCourier` , `BikeCourier` , `CarCourier` , у каждого своя скорость доставки.

8. Контрольные вопросы

1. Что такое наследование?
2. Что такое базовый класс?
3. Что такое дочерний класс?
4. Зачем нужно наследование?
5. Как записать, что класс наследуется от другого класса?
6. Что такое переопределение метода?
7. Для чего нужен `super()` ?
8. Почему не нужно копировать одинаковый код в несколько классов?
9. Как понять, подходит ли наследование?
10. Почему “воин является юнитом” — хорошая связь для наследования?
11. Почему слишком глубокое наследование может быть плохим?
12. Можно ли хранить разных наследников в одном списке?

9. Краткий конспект в конце

Главная идея темы: наследование помогает вынести общий код в базовый класс, а отличия оставить в дочерних классах.

Базовая конструкция:

```
class Unit:
    pass

class Warrior(Unit):
    pass
```

Пример с `super()`:

```
class Unit:
    def __init__(self, name):
        self.name = name

class Warrior(Unit):
    def __init__(self, name, armor):
        super().__init__(name)
        self.armor = armor
```

Переопределение метода:

```
class Warrior(Unit):
    def attack(self):
        print("Атака мечом")
```

Важно запомнить:

- наследование используется для похожих классов;
- общее выносится в базовый класс;
- отличия пишутся в дочерних классах;
- `super()` вызывает код базового класса;
- дочерний класс может переопределить метод;
- наследование должно читаться как “является”.

Частые ошибки:

- копировать одинаковый код;
- забывать указать базовый класс;
- забывать `super()`;
- наследовать неподходящие сущности;
- случайно переопределять методы;
- строить слишком сложные цепочки наследования.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать базовый класс;
- создавать дочерний класс;
- понимать, какие данные лучше вынести в базовый класс;
- использовать наследование для похожих сущностей;
- переопределять методы;
- использовать `super()` в `__init__`;
- создавать списки разных наследников;
- объяснять, когда наследование подходит, а когда нет;
- избегать дублирования кода через наследование.